

Semantic Structure in Object-Centric Discrete Representations for Policy Learning

Rohan Menon

Vrije Universiteit Amsterdam, De Boelelaan 1105, 1081 HV Amsterdam

Abstract. Discrete representation learning has advanced rapidly, yet the semantic content of the learned codebooks is rarely examined. It remains largely unknown whether individual tokens can correspond to interpretable concepts such as “the agent is carrying the key” or “the door is open”, and under what conditions this alignment can emerge without explicit supervision.

This thesis investigates these questions by proposing an object-centric VQ encoder with a shared Transformer backbone, this encoder is trained purely on observations from a grid-world environment solely using reconstruction loss (no reward signal or semantic labels provided). Three independent VQ readout heads, each head dedicated to a specific tracked object, are then evaluated on their semantic alignment with their learned encoding for their object. Logistic regression probes trained against the ground-truth environment labels reveal that the key head achieves 0.997 probe accuracy for the carrying key state, where four tokens respond specifically to the state of carrying the key. The object-centric architecture determines which cells each head reads; reconstruction training alone then produces crisp semantic partitioning within each head’s codebook, without reward signal or label supervision. To investigate the applicability of these learned codes to downstream policy learning a reinforcement learning policy was trained on the fully discrete representation, this policy converged across all tested seeds approaching the performance of the raw observation baseline. However, this approach demonstrates no clear improvement in sample efficiency, with interpretability as the primary clear advantage.

These results suggest that object-centric codebook factorisation is a useful inductive bias for learning grounded discrete representations, and research toward neurosymbolic integration and world-model planning are potential directions for further exploiting this compact, discrete semantic structure.

Keywords: Reinforcement Learning · Discrete Representation Learning · Vector Quantization · Object-Centric Representations · Semantic Interpretability · World Models

1 Introduction

1.1 Context and Motivation

Deep reinforcement learning (RL) has proven to be able to produce agents that are capable of strong performance across a variety of challenging domains, from Atari games [15] to complex robotic manipulation [11]. However, this performance comes with the consequence that agents trained directly from high-dimensional observations require millions of interactions with the environment to learn reliable structure and encode knowledge of their environment. This sample inefficiency represents a challenge for RL algorithms especially in environments where data collection is expensive or unsafe.

Therefore, a parallel research line proposes to address this sample inefficiency in RL environments by first learning a compact latent representation of the environment, over which a policy can be trained more efficiently. Hafner et al. [3, 4] introduced Dreamer, which learns a world model in a continuous latent space and uses it to simulate future experience, improving sample efficiency over model-free methods and establishing latent-space planning as a compelling alternative to traditional RL.

Additionally, research in cognitive science has proposed another reason for structured representations. Fang and Sims [2], demonstrate that humans learn compact, abstract state representations under an efficient coding principle. The idea that discrete representations could provide the basis for generalization in biological systems is supported by this research. These two concepts create a clear research gap for the improvement of robust RL systems. While cognitive research suggests that discrete, internal semantic structures may be a characteristic of generalizable learning, world models show that learning over a latent space may increase the sample efficiency of agents. Therefore, combining these two concepts to study agents that learn on discrete encoded relationships whose latent representations carry semantic structure is a natural progression motivated by these areas of research.

An important contribution to this field of research is the Vector-Quantised Variational Autoencoder (VQ-VAE) [20], which demonstrated that neural networks can learn finite discrete codebooks to reliably compress complex visual input. Subsequent work extended this concept to hierarchical generation [16] and simplified quantisation methods [12]. The applicability of VQ-style discretization to RL environments has been demonstrated by Micheli et al. [13], whose IRIS model, a Transformer world model operating over discrete observation tokens, achieves state-of-the-art sample efficiency on the Atari 100k benchmark. However, what both of these methods share is evaluation based solely on task reward. The semantic structure of the learned vocabulary (whether individual tokens correspond to interpretable concepts such as “the door is open” or “the agent is carrying the key” is not investigated.

This gap is especially relevant for several reasons. Semantically grounded tokens could provide a useful basis for learning policies in complex environments, where the discrete structure could be used to help inform faster convergence

or better results. Additionally, these tokens could provide a useful primitive for neurosymbolic or logic-based approaches where this discrete structure can be exploited. There is also increasing research into developing safe AI systems which have increased interpretability and alignment, tokens that correspond to semantic concepts could offer a path to building robust systems that are more trustworthy and explainable. The developments in powerful transformer architectures and well established discretisation techniques creates an opportunity to investigate this gap by observing if semantically aligned discrete representations can be learned from observation through reconstruction without explicit supervision.

1.2 This Thesis

To address this gap, this thesis explores how a transformer-based encoder can be trained to create a codebook of discrete, semantically grounded tokens trained on observations with a continuous RL environment. The goal is to analyze how well these learned representations correspond with semantically relevant states of the environment entirely without supervision. The encoder is solely trained on reconstruction and therefore, does not obtain any reward signal or particular semantic labels, therefore, any learned semantic alignment must arise solely from the inherent structure of the task environment.

Main research question:

Can a Transformer encoder trained purely on observation reconstruction learn a discrete vocabulary that is both semantically interpretable and sufficient for downstream policy learning?

This question has two parts: whether semantic alignment can arise solely from reconstruction training, and whether these learned representations can support reliable policy learning. These are addressed subsequently by the sub-questions below.

Sub-questions:

- Under what conditions do VQ codebooks spontaneously develop semantically grounded structure from unlabelled observations?
- What information does a fully discrete representation require to support reliable policy learning in a sparse-reward environment?

2 Related Work

This chapter examines three relevant research bodies that this thesis is based upon and frames this current contribution in relation to prior work. The aim is to outline where existing methods succeed and where the specific question of semantic alignment within discrete RL representations has been left open.

2.1 Discrete Representation Learning

The foundational addition to the field of discrete world-model learning, and therefore discrete representation learning is the Vector-Quantised Variational Autoencoder (VQ-VAE) of van den Oord et al. [20]. VQ-VAE proposed replacing the continuous latent space of a standard VAE with a learned finite codebook of embedding vectors. During the forward pass, each encoder output is mapped to the nearest codebook entry. This is then used as input to the subsequent decoder. Because this nearest neighbour lookup operation is non-differentiable, gradients cannot flow through this step. In order to overcome this a straight-through estimator is used, where gradients that arrive at the quantised representation are sent directly back to the encoder output, skipping the discontinuous lookup process.

However, a persistent challenge in VQ-VAE training is codebook collapse. This is when only a small subset of codes receive gradient updates, leaving the majority of codes in the codebook unused and the overall representational capacity underutilised. The original paper’s Appendix A proposed an Exponential Moving Average (EMA) update rule that separates the codebook updates from the initial optimisation step. Each codebook vector is updated as a running average of encoder outputs assigned to it rather than purely through gradient descent. Additionally, a complementary dead-code restart mechanism reinitialises underused entries by copying the embedding of a randomly selected active code, amending the codebook to encourage coverage. Together these two methods have been proven to reduce collapse, therefore resulting in increased utilisation of the entire codebook. VQ-VAE-2 [16] extends this approach to hierarchical multi-scale representations. More recently, Finite Scale Quantization (FSQ, [12]) propose a new method that avoids explicitly maintaining the codebook entirely and bounds each dimension of the latent to a small integer grid, and relying on this implicit discretization. FSQ proposes trading codebook flexibility for increased simplicity during training and demonstrates comparable reconstruction results. Earlier work on disentangled continuous representations, particularly β -VAE [7] also act as a basis for this thesis by outlining that architectural biases can encourage structured, interpretable latent spaces unsupervised, an insight that motivates the object-centric factorisation proposed here.

Another line of work investigates object-centric representation learning by using attention-based decomposition rather than quantisation methods. Locatello et al. [21] introduced Slot Attention, a method which maps features to a set of slot representations through an iterative attention mechanism. In this architecture slots specialise to individual objects purely through reconstructive training. Slot Attention proves that object-centric inductive biases can be encoded architecturally, and that the representations they generate can support downstream policy prediction. However, Slot Attention operates continuously generating continuous representations and evaluating the semantic content through supervised prediction heads over whether individual representation units correspond to specific, discrete concepts. This thesis shares this object-centric motivation but in-

investigates whether a discrete vocabulary can develop crisp semantic partitions which are readable without a supervised head.

The key difference between all the aforementioned approaches and this proposed approach is that historically evaluation is measured based on reconstruction quality and generative fidelity. Whether or not the learned codes align with semantically meaningful concepts is yet to be examined.

2.2 World Models and Latent-Space Policy Learning

The Dreamer family of world models [3, 4] demonstrated that both learning and planning over a structured latent space can substantially improve sample efficiency over model-free approaches. This finding directly motivates this thesis’s question of whether a discrete, semantically grounded latent vocabulary can offer further advantages. DreamerV1 [3] develops a Recurrent State Space Model (RSSM) featuring continuous stochastic latents and fully trains a policy through imagined rollouts, reaching competitive results on DMControl with significantly fewer interactions with the environment. DreamerV2 [4] improves on this finding by integrating discrete categorical latents with continuous recurrent states, showcasing competitive performance in Atari at 200k steps. DreamerV3 [5] extends its application to various domains by ensuring stable training across different scales; although it includes discrete elements, it is not solely a discrete model and is thus referenced here for thoroughness rather than for direct comparison.

The most relevant model-based research is IRIS [13], which integrates a VQ tokeniser with a Transformer world model, attaining state-of-the-art results on the Atari 100k benchmark. IRIS shows that discretization enhances RL beyond just compression; the organized vocabulary allows for efficient Transformer modeling of transition dynamics. TD-MPC2 [6] adopts a complementary strategy, acquiring a latent world model for continuous control through temporal difference learning without using explicit quantization. The previous Model-Based RL for Atari [9] demonstrated that acquired environment models can enhance sample efficiency in settings with discrete actions. Sequence modeling methods like Decision Transformer [8] utilize Transformer architectures in offline RL, whereas generalist agents [17] showcase cross-task generalization through token-based representations. In all of these approaches, assessment is carried out solely on task reward and sample efficiency indicators. The internal organization of the acquired representations, particularly, if separate tokens relate to important environmental states, is never assessed. This thesis fills that gap by applying a linear probe methodology directly to the VQ codebooks within the controlled RL environment.

2.3 Interpretability of Learned Representations

Alain and Bengio [1] introduced the application of linear classifiers for evaluating the semantic content of intermediate representations in neural networks, demonstrating that the performance of a logistic regression probe, trained on the activations of a layer, offers a reliable estimate of the information encoded

by that layer. This "probing" approach has since turned into a standard technique in representation evaluation. In this context, logistic regression examines one-hot token indices to determine if the discrete codebook accurately reflects the true environment labels without any supervised labeling during training.

A related area of research aims to create interpretable representations by design. Concept Bottleneck Models [10] require that intermediate representations align with predefined human-understandable concepts, allowing for intervention during testing. Although this method requires labeled concepts during training, it demonstrates that aligning representations with semantic ideas is feasible and beneficial in practice. Doshi-Velez and Kim [19] present a comprehensive framework for assessing interpretability in machine learning, differentiating between application-grounded, human-grounded, and functionally-grounded evaluation standards; the linear probe approach utilized in this thesis is classified under the functionally-grounded criteria.

This thesis is unique from the aforementioned works by utilizing probe-based analysis specifically for VQ codebooks within a reinforcement learning framework, focusing not on whether a continuous representation contains semantic information but rather on whether the discrete codebook indices retain such information post-quantization. Additionally, the object-centric factorization; distinct VQ heads for every tracked object, introduces an architectural inductive bias aimed at semantic alignment that, to the authors' knowledge, has not been previously assessed with linear probes in a reinforcement learning context. No previous studies have methodically investigated if specific VQ codes in a reinforcement learning encoder relate to semantic states of the environment, or if object-centric codebook factorization enhances this correlation. This thesis tackles both questions.

3 Methodology

3.1 Encoder Architecture

The encoder follows an object-centric design. There is a shared transformer backbone which process the full observation, three independent VQ readout heads each quantise a different aspect of the scene (Figure 1). This factorisation is the central architectural addition and is motivated by the hypothesis that linking dedicated codebook capacity to individual objects can help promote semantically grounded tokens.

The encoder receives the partial grid observation as input (outlined in section 3.5). Each cell's three integer channels (object type, colour, state) are mapped through separate learned embedding tables and projected into a $D_{\text{model}} = 64$ dimensional space. A learnable CLS token is added to the beginning of the sequence and positional embeddings are added, yielding a sequence of 50 vectors processed by a two-layer, four-head Transformer encoder.

The three VQ readout heads operate as follows. The agent head takes the CLS token output and passes it through a VQ layer with $K = 64$ codebook

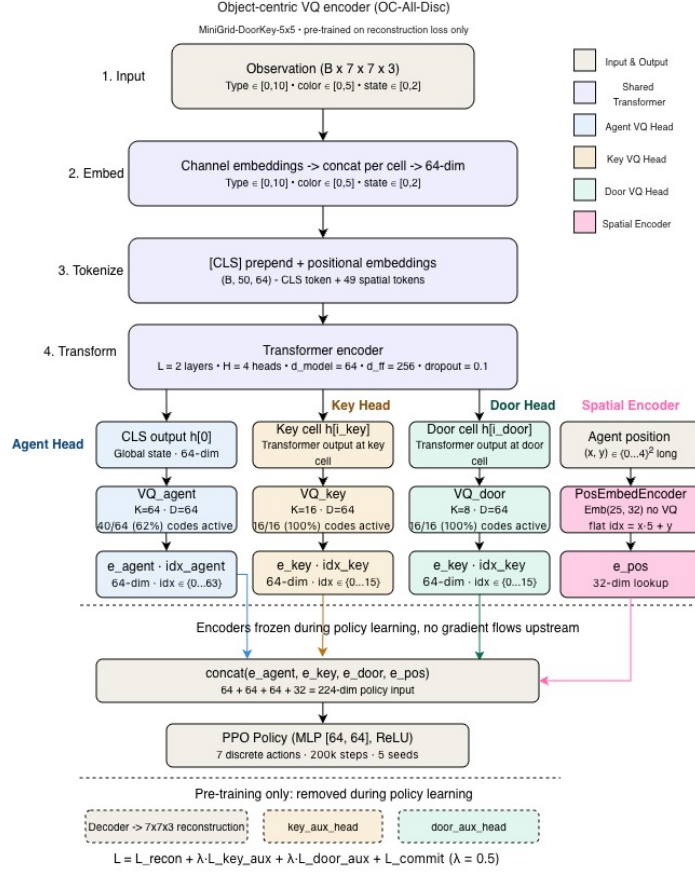


Fig. 1. Overview of the object-centric VQ encoder (OC-All-Disc).

entries, producing e_{agent} . The key and door heads read the transformer’s output at the spatial cell corresponding to the key’s and door’s current grid position and applies a VQ layer with $K = 16$ and $K = 8$ respectively, yielding e_{key} and e_{door} . In this approach, cell positions are detected from the observation’s type channel at runtime. When an object is absent from the partial observation (for instance, when the key has been picked up), the corresponding head falls back to a learned absent embedding. This fallback is itself a reliable signal as the absent-embedding activation is tightly correlated with the `carrying_key` state.

Each VQ layer selects the nearest codebook entry by L_2 distance:

$$k^* = \arg \min_k \|z - e_k\|^2. \quad (1)$$

Gradients are propagated through this discrete lookup process by the use of a straight through estimator. Codebook entries are updated using EMA rather

than gradient descent, in addition with a dead-code restart that reinitialises unused codes to help prevent codebook collapse. The commitment loss $\beta \|z - \text{sg}(e_{k^*})\|^2$ (with $\beta = 0.10$) encourages encoder outputs to remain close to their assigned entry.

The four elements: e_{agent} (64-dim), e_{key} (64-dim), e_{door} (64-dim), and e_{pos} (32-dim) are then concatenated to form the 224 dimensional full policy input. The positional embedding e_{pos} encodes the agent’s discrete cell positional index to a 32-dimensional trainable vector, this step does not require quantisation as the position space is already discrete. Reconstruction and per-head auxiliary decoders are only used during pre-training and removed during policy learning. Full hyperparameter settings are listed in Appendix A

3.2 Object-Centric Encoder Pre-Training

The encoder is trained with the random-policy dataset of 59,697 observations mentioned in Section 3.1. The goal during training is to minimize cross-entropy reconstruction loss. No reward signals, environmental labels, or task-specific information are utilized. Therefore, any semantic separation that arises is derived from the structure of the observations themselves. The encoder employs Adam as the optimizer ($\text{lr} = 3 \times 10^{-3}$) for 80 epochs, utilizing a batch size of 128. EMA codebook updates and dead code restart techniques are utilized during every forward pass as mentioned in Section 3.1.

An initial approach used gradient-based codebook updates (this contained a learnable embedding layer updated by backpropagation) and a single CLS-token head with $K = 32$. This approach suffered clear codebook collapse, after training there were just 11 of 32 codes active. This mechanism is a common feedback loop characteristic generated by VQ training as popular codes attract from assignments, receive more gradient updates and therefore become more central, while unused codes drift further from the distribution. The skewed distribution of random-policy observations exaggerates this effect. Switching to EMA updates with an increased codebook size increased the utilisation of the codebook codes to 57/64, and using the object-centric three-head design with codebooks sizes of $K = 64, 16$, and 8 achieved full codebook utilisation on the key and door heads (100% active) and 62% utilisation on the agent head. The full training history of this progression is detailed in Section 4.1; hyperparameters are listed in Appendix A.

3.3 Policy Conditions

This section contains the policy conditions that were investigated in this study. The specific hyperparameters used for reproducibility are included in Appendix A. Five conditions are considered, ranging from the raw-observation baseline where the flattened observations are fed into a flat MLP to the proposed fully discrete representation. Each condition uses the same PPO training configuration [18] with a frozen encoder, the encoder weights are frozen after pre-training and not updated during policy learning. The network used by the PPO policy is a

Table 1. Policy conditions evaluated in this study.

Condition	Input description	Dim	Notes
Raw observations	Flattened $7 \times 7 \times 3$ obs (integers)	147	Upper-bound baseline
OC-Agent	CLS-token VQ embedding only (e_{agent})	64	Ablation: no spatial or object context
OC-All	Concat[$e_{\text{agent}}, e_{\text{key}}, e_{\text{door}}$]	192	Three semantic tokens, no position
OC-All+Pos	Concat[$e_{\text{agent}}, e_{\text{key}}, e_{\text{door}}, \text{raw_xy}$]	194	Oracle diagnostic: float (x, y) appended
OC-All-Disc	Concat[$e_{\text{agent}}, e_{\text{key}}, e_{\text{door}}, e_{\text{pos}}$]	224	Proposed: fully discrete with pos embeddings

two hidden layer MLP of size [64, 64] with ReLU activations in between. These five conditions are summarised in table 1.

3.4 Downstream Policy Learning

The five policy conditions use a shared downstream learning setup. After the encoder has been pre-trained as referenced by Section 3.2, the weights are frozen during policy learning. The encoder practically serves as a fixed feature extractor, at each environment step it maps the raw observation to a representation vector, this is then passed as input to the policy network.

The policy optimisation uses Proximal Policy Optimisation (PPO) [18]. Where the policy network is an MLP with two hidden layers [64, 64] with ReLU activations, with separate output heads for the action logits and value estimates. The input dimensionality for the network vary based on the dimensionality of the encoders representation vectors, depending on the condition as specified in Table 1: from 147-dim (Raw) to 224-dim (OC-All-Disc). The other network and optimiser hyperparameters are kept constant to ensure that the differences in runs are solely attributable to the quality of the input encoding. Full PPO hyperparameters are listed in Appendix A.

Each condition is trained for 200,000 total environment steps across 5 different independent random seeds. The policy is evaluated every 5,000 steps by running 10 rollouts and recording the mean episodic return each time.

3.5 Evaluation

There are three evaluation metrics used in this study, each corresponding to three components of the experimental design: codebook quality, semantic alignment and policy learning.

The quality of the codebook is measured by using two criteria computed over the encoder’s training dataset. The active code count represents the number of codebook entries that are assigned to at least one observation in the dataset, entries with no assignments are considered collapsed and therefore inactive. Perplexity is calculated as $\exp(H)$ where H represents the entropy of the per-code

usage distribution. Normalised perplexity is calculated as $\exp(H)/K$ where H represents the entropy of the per-code usage distribution. A value of 1.0 indicates perfectly uniform usage across all K codes, whereas values approaching 0 indicate that a small number of codes dominates the representation, making heads with different codebook sizes directly comparable.

To measure the semantic alignment of the learned tokens, logistic regression probes are used [1]. For each of the VQ heads, the one-hot token index (K dimensional binary vector) is used as the feature for the logistic regression classifier to predict each of the binary ground truth labels (`door_open`, `door_locked`, `carrying_key`). Probes are trained and fit on an 80/20 train-test split. Probe accuracy significantly above the majority baseline shows that the token assignments contain relevant information about their corresponding semantic concept. In addition to the per-head logistic probes there is a combined probe which is fit on the concatenation of the one-hot encodings for each of the three heads. The per-token purity is calculated as $P(\text{label} = \text{True} \mid \text{token} = k)$ for each code k , identifying individual codes that prove they act as specific semantic indicators.

The policy learning performance is measured by the mean episodic return (which is a downstream evaluation), which is averaged across 10 deterministic test episodes run every 5,000 steps in the environment. Training continues for 200,000 total environment steps with the PPO policy [18]. Experiments are then repeated across 5 independent random seeds. Results are given as the mean $\pm$ standard deviation of the final evaluation run across the various seeds, measured alongside the mean peak return per seed and the bootstrap rate (the proportion of seeds that achieve a mean return above 0.5 at any point during training).

3.6 Environment and Data

The proposed architecture (OC-All-Disc) was tested in MiniGrid-DoorKey-5x5 [14], a simple grid-world environment in which an agent must complete a sequential set of tasks: locate a key, pick it up, use it to unlock a locked door, and navigate to the goal cell. This specific environment was chosen for two primary reasons. Firstly, it provides access to ground-truth boolean state labels (`door_open`, `door_locked`, and `carrying_key`) that are not derivable from raw pixel values themselves. These are not available to the agent during training or during downstream testing to enforce the unsupervised nature of the encoding, however, they are used to directly assess semantic alignment. Secondly, the environment is small enough to run a comprehensive ablation study under a fixed computational budget while still containing enough of a non-trivial structure that requires multiple sub-goals.

The observation space is a $7 \times 7 \times 3$ integer-valued grid, representing a partial view centred on the agent. Each of the 49 cells encodes an object type, colour and state in three integer channels. The agent is not able to observe the entire 5×5 environment directly, as cells outside it’s view are masked. The action space consists of seven discrete actions: turn left, turn right, move forward, pick up, drop, toggle, and done. The reward function is very sparse, the agent receives a

Table 2. Codebook utilisation after 80 epochs of encoder pre-training.

VQ Head	K	Active codes	% active	Perplexity/ K
Agent (CLS token)	64	40	62%	0.32
Key (cell readout)	16	16	100%	0.72
Door (cell readout)	8	8	100%	0.61

reward only when reaching the goal cell, this is then discounted by the number of steps taken. All the other transitions yield no reward.

To gather training data for the encoder a uniformly random policy was run for 300 episodes of up to 100 steps each, yielding 59,697 observations. This random policy data is specifically chosen to provide a variety of environment states without directly exploiting the task structure or providing any implicit supervision. Ground-truth labels (`door_open`, `door_locked`, `carrying_key`) are recorded at each step from the environment state and are used exclusively for semantic alignment evaluation; they are never provided to the encoder during training.

4 Results

This chapter outlines the findings of the various experiments across three primary factors: encoder pre-training/codebook utilisation, semantic alignment of the learned vocabulary and downstream policy learning performance. Each of these subsections address one of these components and therefore elements of the overarching research question. Building toward a clearer picture of whether or not object-centric discrete representations have semantic grounding and can be used for policy learning.

4.1 Encoder Pre-Training

The object-centric encoder was trained for 80 epochs on the 59,697-observation dataset. Reconstruction loss converged across all heads, indicating that the Transformer backbone successfully learned to represent the observation content. Table 2 summarises the codebook utilisation for the final encoder, and Figure 2 shows the full training curves for losses and per-head perplexity.

The key and door heads achieve full codebook usage, every code in those codebooks is used at least once and the normalised perplexity of the key head (0.72) indicates quite well distributed usage of codes. The lower normalised perplexity of the door head (0.61) reflects the binary nature of the door state (locked vs open): as shown in Figure 3 a small number of codes account for the majority of assignments. This is consistent with the semantic alignment results in Section 4.2 where a single token is able to account for all door-open observations, while the remaining seven codes separate the locked state. The agent head is less utilised at 62% (40/64 active codes) which is expected as it likely reflects the non-uniform distribution of agent positions in the random policy dataset.

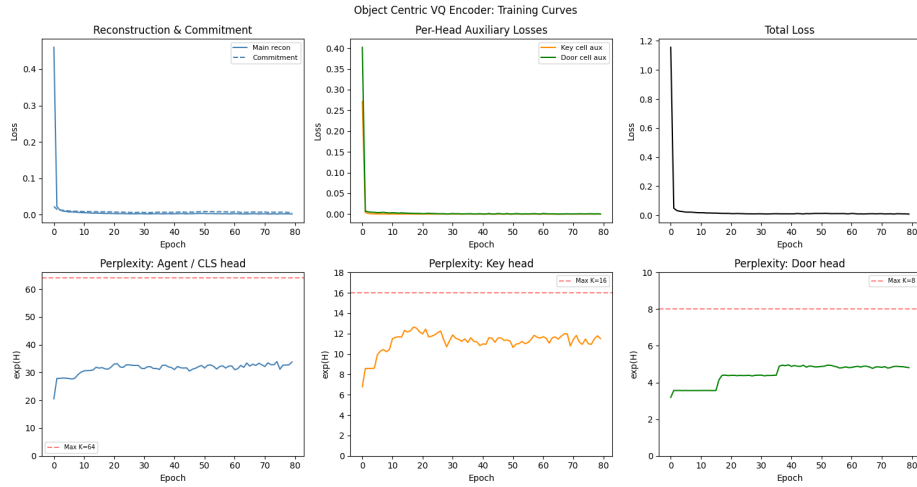


Fig. 2. Object-centric VQ encoder training curves over 80 epochs. Top row: reconstruction loss, per-head auxiliary losses, and total loss all converge rapidly. Bottom row: perplexity ($\exp(H)$) for each VQ head, with dashed red lines marking the maximum possible perplexity (K). The key and door heads converge well indicating strong codebook utilisation.

Figure 3 shows the token assignment distribution for each head, confirming full usage on the key and door heads and depicting the skewed but non-collapsed distribution of the agent head.

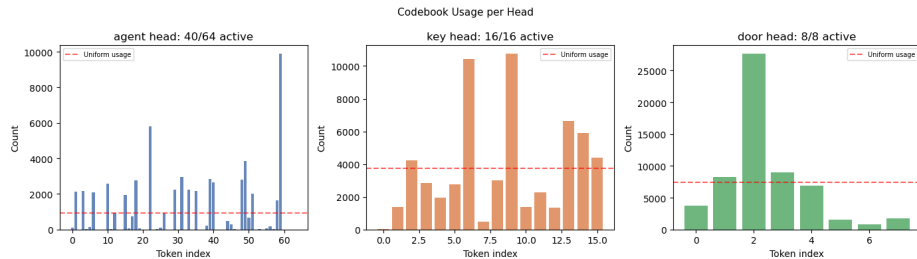


Fig. 3. Codebook usage per head. Bars depict the assignment count for one codebook token over the full training dataset; the dashed red line indicates uniform usage.

These results represent a major improvement over the initial encoder prototype. The first design used a single CLS-token readout head with $K = 32$ and gradient-based codebook updates. After training, only 11 of 32 codes (34%) were active, with the remaining 21 codes never used. Replacing gradient-based updates with EMA codebook updates and adding dead-code restart raised active

codes to 57/64. However, this approach still lead to issues during policy exploration as the spatial bottleneck was too overwhelming as one CLS token could not represent the relevant agent history, positional information and semantic context. The subsequent redesign to an object-centric three-head architecture, with smaller per-object codebooks tuned to the scale of each concept ($K = 64$ for the agent, 16 for the key, 8 for the door), achieved the per-head codebook usage reported above.

4.2 Semantic Alignment

Table 3 outlines the logistic regression probe accuracy for each head/label combination, shown alongside the majority class baseline and the combined probe fit on the concatenated one-hot encodings of the three heads. Figure 4 visualises these results as grouped bar charts.

Table 3. Probe accuracy (logistic regression) for each VQ head and ground-truth label. Bold denotes the best per-head result for each label.

Label	Majority	Agent	Key	Door	Combined
door_open	0.944	0.970	0.949	0.972	0.985
door_locked	0.925	0.967	0.933	0.964	0.978
carrying_key	0.665	0.764	0.997	0.675	0.997

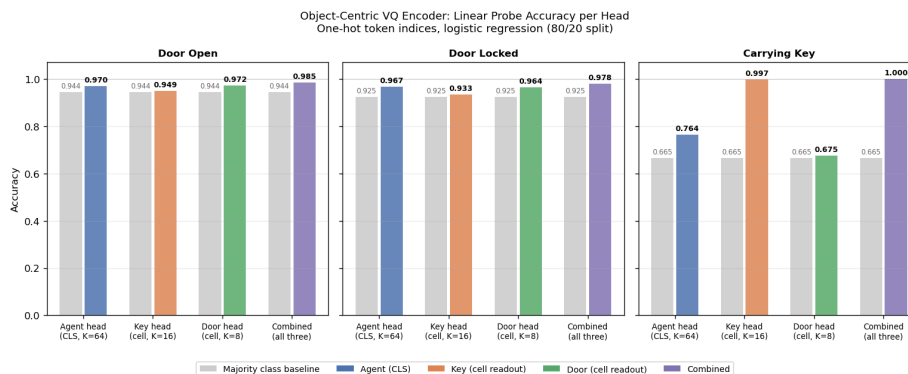


Fig. 4. Linear probe accuracy per VQ head for each ground-truth label. Grey bars represent the majority-class baseline. The key head (orange) achieves near-perfect accuracy for **carrying_key** (0.997), far exceeding the baseline of 0.665. The combined probe (purple) achieves the highest accuracy across all three labels.

The door head also achieves almost perfect semantic alignment for the door state labels. Token 5 of the door head is shown to have $P(\text{door_open} = \text{True}) =$

1.000 while all other tokens have $P(\text{door_open} = \text{True}) \approx 0.000$. The door head therefore has learned a single dedicated vocabulary for the open state, while the other remaining seven codes represent the locked state and it’s various sub-variants. The linear probe results confirm this finding has the accuracies are 0.972 for `door_open` and 0.964 for `door_locked`, meaningfully above the majority baselines of 0.944 and 0.925.

Figure 5 shows the per-token conditional probability $P(\text{label} = \text{True} \mid \text{token} = k)$ for the key and door heads, sorted by this probability. There are four key head tokens which cluster around $P = 1.000$ and the remaining twelve tokens cluster around $P \approx 0.000$, with no ambiguous intermidate codes. This structural partition, is the primary finding in relation to the first research sub-question. Demonstrating that the codebook has learned to encode and cluster the semantic concepts as specific vocabulary regions rather than softer distributions.

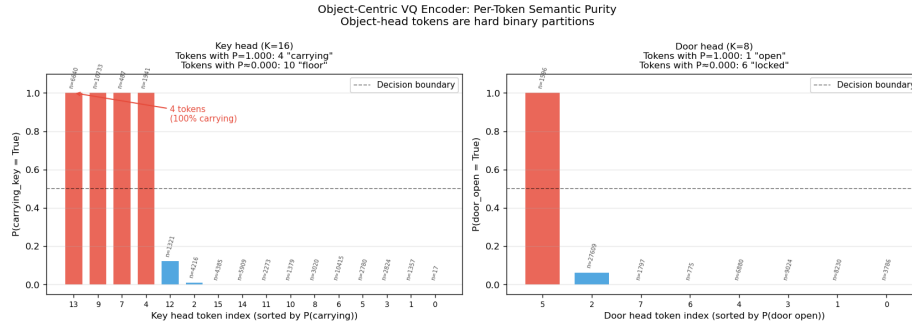


Fig. 5. Per-token semantic purity for the key and door head. Each bar shows $P(\text{label} = \text{True} \mid \text{token} = k)$ for tokens sorted by this probability. Red bars indicate tokens with $P = 1.000$, blue bars indicate tokens with $P \approx 0$.

The agent’s head accuracy on `carrying_key` (0.764) is notably lower than the key head’s (0.997), despite the agent head having access to the same underlying representations and observation. This difference is explained by the architecture. The agent head takes the CLS token as input, which aggregates information that is gathered from all 49 spatial positions through the Transformer’s attention mechanism. The key-carrying state, however, is a local event that occurs just at the key cell. This signal is diluted when it is pooled with the representations of all the other cells. The key head contrastingly directly reads the output of the Transformer at the key’s own spatial position, or activates the absent embedding when the key is not visible, which itself is it’s own reliable carrying signal. This comparison outlines the value of the object-centric design, where global pooling loses the local semantic signal that direct object-cell readout maintains.

The combined linear probe achieves 0.985 for `door_open`, 0.978 for `door_locked`, and 0.9997 for `carrying_key`, confirming that the three heads capture complementary and largely non-redundant information about the environment state.

4.3 Policy Learning Efficiency

Table 4 outlines the policy learning results for all five conditions, across five random seeds. Figure 6 shows the full learning curves.

Table 4. PPO policy learning results at 200k environment steps (5 seeds).

Condition	Dim	Mean return $\pm$ std	Bootstrap	Peak/seed
Raw observations	147	0.965 ± 0.001	5/5 ($\sim 30k$)	0.967
OC-Agent	64	0.000 ± 0.000	0/5	0.000
OC-All	192	0.116 ± 0.188	2/5	0.174
OC-All+Pos	194	0.307 ± 0.350	4/5	0.521
OC-All-Disc	224	0.768 ± 0.158	5/5 (45–85k)	0.945

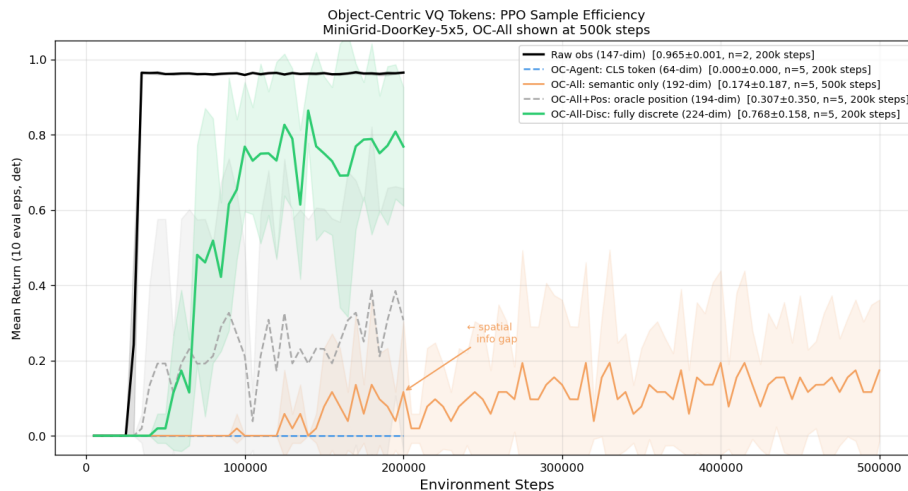


Fig. 6. PPO learning curves for all five conditions on MiniGrid-DoorKey-5x5. Mean return per episode plotted against environment steps, averaged over 5 seeds with shaded ± 1 standard deviation regions.

The ablation process reveals a clear outline about the information requirements for this task. Figure 6 outlines the policy learning across steps for each of the tested models.

OC-All-Disc (0.768 ± 0.158 , 5/5 seeds, mean peak 0.945) is the proposed method and the best-performing non-raw condition. By replacing the two float coordinates with a 32-dimensional pre-trained positional embedding it yields a further $2.5\times$ improvement over the float condition with purely discrete inputs. Individual seeds reach peak returns of up to 0.970, approximately matching the raw-observation ceiling of 0.967.

OC-All+Pos (0.307 ± 0.350 , 4/5 seeds) adds the current agent position concatenated as two raw float values (x, y coordinates). This instantly creates a $2.6\times$ improvement over OC-All which confirms the spatial bottleneck as the limiting factor to policy performance.

OC-Agent (0.000 mean return, 0/5 seeds) fails completely: a single CLS-token semantic embedding without spatial information is unable to provide any useful signal for navigation. This is despite the fact that it is able to learn a useful embedding of the semantic concepts in the environment. OC-All (0.116 ± 0.188 , 2/5 seeds) improves slightly, however, still primarily fails at policy learning due to this same spatial bottleneck. OC-All (orange) is shown up to 500k steps to illustrate its slow, partial convergence and the spatial bottleneck that has been reached.

The primary issue here is that both OC-Agent and OC-All produce representations that are not specific to a certain position, the same token sequence can occur regardless of where the agent is in the grid. The policy, therefore, receives fixed-length vectors which are required to be mapped to actions, these can be identical inputs in situations that receive entirely different actions depending on the location of the agent. For example, the token sequence encoding “key carried, door locked” is equally likely when the agent is adjacent to the door needing to open it, or on the opposite side of the grid needing to navigate toward it first. During training, the gradient from these different situations pushes the policy’s weights into conflicting directions, since this conflict occurs at different semantic states for each agent position the updates cancel and no action mapping can be effectively found. This directly contrasts the raw observation baseline where the agent’s position is implicitly understood from the pixel data, the same observation never requires contradictory actions.

5 Discussion

The results demonstrate that semantically grounded discrete representations can arise from training purely on reconstruction. However, their practical performance when applied as a basis for an RL policy depends heavily on the auxiliary information surrounding it. This chapter discusses the impact these findings have on the conditions for semantic alignment, information requirements for downstream reinforcement learning and broader implications for interpretable discrete reinforcement learning.

5.1 Codebook Quality and the EMA Fix

The first sub-research question asks under which conditions VQ codebooks are able to spontaneously develop semantically grounded observations from unlabelled data. The initial codebook collapse present by the original gradient-descent encoder approach outlines that having healthy codebook utilisation is a requirement for being able to encode any semantic structure at all. In the

initial codebook only 11/32 codes were active which made it very difficult to partition the observation space into meaningful concepts, as the vocabulary itself is too limited to support distinct semantic concepts.

The EMA update rule [20] addresses this challenge by separating updates to the codebook from the gradient descent optimiser. Instead of updating codes proportionally to how often they are chosen (which creates the positive feedback loop responsible for collapse). EMA replaces each entry with a running average of the encoder outputs assigned to it. Additionally, the dead code restart makes it so that unused codes are able to be reinitialised into active regions of the codebook space and can be reintroduced to training. The impact of these two techniques raised codebook utilisation from 11/32 (34%) to full utilisation for the key and door heads, creating the codebook health needed to effectively try and encode the vocabulary.

This has a broader effect for discrete representation learning, as the conditions that are required for semantically grounded structure to emerge are not solely reliant on architecture or data, but also on the stability of the quantisation process itself. Without these updates the feedback dynamics of the VQ process would prevent enough codebook coverage for distinct semantic concepts to be encoded in distinct regions of the vocabulary.

5.2 Semantic Alignment

The main result found in relation to the second sub-question is that a combination of object-centric codebook factorisation and spatial cell readout creates the conditions for semantic partitioning to emerge from reconstruction training alone. As seen in Figure 5 the key head is able to develop a vocabulary where four codes exclusively appear when the agent is carrying the key. The remaining twelve appear solely when the key is on the floor. This binary partition without any intermediate tokens was achieved without direct label supervision.

This crisp partition is directly a result of the object-centric design. While global CLS pooling is able to encode `carrying_key` to approximately 0.85 probe accuracy, this global pooling results in softer statistical associations. This is because the carrying signal is diluted across all 49 spatial positions in the grid. Contrastingly, the key head reads the output of the transformer at the key’s spatial cell. When the key is present on the floor of the grid that cell carries a certain representation. Additionally when it is absent it activates the learned absent embedding, which also becomes a specific carrying signal. This results in a vocabulary that is structurally separated rather than statistically biased.

This finding connects to the initial cognitive science motivation for this work [2]. The finding that humans develop compact, reusable representations from complex continuous observation spaces is reflected here in the encoders emergence of a small semantically clean vocabulary for each element of the encoders observation space. The fact that a structure with similar compact, semantic properties emerges in a neural encoder trained purely on reconstruction, suggests that object-centric inductive biases may be a useful principle for learning grounded discrete representations in a broader context, beyond this specific environment.

The results from the learned codebook from the door head, are consistent with this picture. The gain was smaller over the majority baseline best explained by the infrequent nature of the door-open state in this random-policy dataset, rather than an inherent failure of the approach. The combined probe still achieves 0.985 for `door_open` against the majority baseline of 0.944 confirming that the tokens do carry additional information about the door state.

5.3 Policy Learning

The second research sub-question asks what information a fully discrete representation requires to support stable policy learning. The results are clear, semantic tokens alone are insufficient. Positional information is necessary, and the quality of the positional encoding is as important as its presence.

The inability for a policy to learn based on the encodings of OC-Agent (0.000 mean return, 0/5 seeds) and the low performance of OC-All (0.116, 2/5 seeds) both highlight the same fundamental issue. Semantic tokens are able to encode *what* objects are and their current *state*, however, they carry no information about *where* the agent is in relation to these objects. As the same sequence of semantic tokens can occur at various positions in the grid, the policy receives identical inputs for situations that require varying actions depending on the location of the agent. This creates noisy and contradictory policy gradients that harm convergence in training. The policy struggles to learn consistent mappings to actions when the same observation representation can correspond to multiple required behaviors across episodes.

OC-All+Pos (0.307, 4/5 seeds) confirms this spatial bottleneck by doubling the accuracy and increasing the number of seeds that are able to bootstrap significantly. However, these underwhelming results also reveal that the *form* that this information is presented in matters. The stronger performance of OC-All-Disc (0.768, 5/5 seeds) over OC-All+Pos despite using only two additional float values shows that the 32-dimensional pre-trained positional embedding provides a far richer signal to the MLP than just raw coordinates. This embedding learns a smooth spatial prior, where nearby positions receive nearby embeddings. It’s dimensional scale is compatible with the VQ embeddings it is being concatenated with. Raw float coordinates lack both these properties while also losing the strictly discrete nature that makes this structure valuable.

All conditions experienced demonstrable training instability. Particularly the tendency of OC-All-Disc seeds to reach almost optimal performance before oscillating downward reflects the sensitivity of PPO policies to learning rates in sparse reward environments. An extension to this implementation would be to implement a cosine or step learning rate schedule to improve training stability and convergence.

5.4 Limitations

Three fundamental limitations confine the conclusions that can be drawn from this study, each one with implications for the context in which the findings should be interpreted in relation to the research questions.

Firstly, the experiments were all conducted within a small environment with clearly valued semantic states. The crisp codebook partition observed in this setting (four codes exclusively for carrying the key, one code exclusively for the open door), may be a feature of the clean state structure of DoorKey-5x5, where semantic concepts are able to map onto discrete observations patterns which do not overlap. Environments with graded, overlapping or varying semantic states may not benefit from the same benefits found by this object-centric architecture. This architecture in those scenarios may produce softer statistic associations rather than the cleaner structural partitions seen here. The natural next evaluation of this encoding architecture would be to test it on environments such as MiniGrid-FourRooms or BabyAI, which feature more objects and more complex states. This would help to more rigorously test whether this semantic partitioning is a property of the approach or a result of the specific environment structure.

Secondly, this proposed approach is proven to not improve sample efficiency over training an MLP over raw pixel observations. OC-All-Disc requires approximately 45-85k steps to bootstrap in comparison to approximately 30k for raw observations and also achieves a lower mean final return (0.768 vs 0.965). The primary advantage achieved by this approach is interpretability rather than raw results or sample efficiency. This contribution is meaningful within the context of trustworthy and explainable AI [19], however the practical case for this approach in situations where sample efficiency is the primary concern needs to be evaluated.

Thirdly, the instability observed during training of OC-All-Disc limits the strength of the conclusions that can be drawn from the policy learning. Five seeds showing high variance (0.768 ± 0.158) and individual seeds oscillating downward after reaching near-optimal performance during training reflect the sensitivity of the PPO policy to the fixed learning rate rather than the strength of the learned representation. The implementation of a learning rate schedule as referenced would help to establish whether the representation itself is the limiting factor.

5.5 Future Work

These results create several directions for future research, two of which are especially clearly motivated by the current findings.

The clearest and most well-motivated extension to this study, is to integrate the object-centric encoder with a Transformer-based world model such as IRIS [13], replacing the VQ-VAE tokeniser presented in the paper with the object-centric heads presented here. This would allow for planning to occur directly over semantically grounded discrete tokens rather than ambiguous codebook entries. The key question to be answered with this integration is whether or not semantically grounded tokens improve the world models ability to predict transitions in this discrete space. Especially for transitions that correspond to specific, discrete events like picking up a key or unlocking a door. IRIS provide

a challenging, evaluation framework for this, with the Atari 100k benchmark offering a good test if this approach scales beyond grid worlds.

The second well-motivated direction for future study is neurosymbolic integration. The crisp partitions observed in the key and door heads show that individual codebook tokens can be mapped to logical concepts. This makes this learned vocabulary a natural way to bridge neural perception and symbolic reasoning. Given that these tokens already encode logical reasoning, it may be beneficial to use a downstream network that can exploit this compressed representation rather than regular policy-gradient methods. An experiment to test this would be replacing the PPO policy with a logic-based planning model that can operate over sequences of codebook tokens. This would help to test whether the symbolic structure is able to support more sample-efficient learning than gradient-based policy learning.

In order to make broader claims surrounding the generalisability of this approach, the evaluation needs to be extended to more complex environments. Environments with more objects, longer task horizons and complex dependencies between states would test the extent to which whether this object-centric factorisation is able to continually produce semantically grounded partitions in situations where the observation space is not as structured. Multi-task generalisation experiments, where a single encoder is trained on one MiniGrid variant and evaluating the learned vocabulary’s transferability to a related task with similar semantic concepts would additionally help test how specific the structure of the codebook is to the task.

Finally, the end-to-end training process of the entire pipeline could be investigated. The current approach of freezing the encoder during policy learning helps to preserve semantic alignment but stops the encoder from being able to adapt to the nature of the downstream task. An approach of joint training with a regularisation term to maintain probe accuracy above a certain level could help the representations specialise for policy learning while keeping their semantic structure. This could help test whether the interpretability and performance can be achieved simultaneously as opposed to a trade-off.

6 Conclusion

This thesis investigated whether a Transformer encoder trained purely on observation reconstruction can learn a discrete vocabulary that is both semantically interpretable and sufficient for downstream policy learning. The findings address each of the two sub-questions in turn.

In relation to the first sub-question (under what conditions do VQ codebooks spontaneously develop semantically grounded structure from unlabelled observations), the findings demonstrate that there are two necessary conditions. Firstly, codebook health must be ensured to make sure the vocabulary is varied enough to be able to generate meaningful semantic partitioning. Techniques such as EMA and dead-code restarts are especially useful for this application. Secondly, an object-centric architecture bias is important, dedicating specific VQ

readout heads for semantic concepts that are intended to be tracked, combined with spatial cell readout helps to create the crisp semantic partitioning which is challenging to find with just global CLS pooling. Under these two conditions the key head achieved 0.997 probe accuracy for `carrying_key`, with four tokens exhibiting $P(\text{carrying_key} = \text{True}) = 1.000$, and the door head learned a single dedicated token for the open state. These partitions emerged from reconstruction loss alone, without reward signal or label supervision, however guided by the object-centric inductive bias written into the architecture. This type of inductive bias is especially useful in applications where the types of semantic concepts are known, and specifically required to be encoded.

Regarding the second research sub-question (what information a fully discrete representations requires to support reliable policy learning in a sparse-reward environment), the study provides a clear answer. For a traditional RL gradient policy approach semantic tokens are important but not solely important. Positional information must accompany these representations, and the quality of it’s encoding is as important as it’s presence. OC-All-Disc, the proposed fully discrete encoder combining semantic tokens with a pre-trained positional embedding achieved a mean return of 0.768 ± 0.158 across five seeds at 200k environment steps, with individual seeds reaching 0.970, reaching the raw observation ceiling of 0.967. However, this approach does not improve sample efficiency over planning over raw pixel observations, the primary advantage is interpretability over data efficiency.

When taken together, these results suggest that semantically partitioned discrete structures can arise from reconstruction-based training given the right architectural conditions. However, closing this gap to raw observation performance in practical policy learning situations requires further development, especially in richer environments. This could come from world-model-based planning, neurosymbolic integration or some model that is able to exploit the rich nature of these learned semantic codes over a standard RL policy.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alain, G., Bengio, Y.: Understanding intermediate layers using linear classifier probes. ICLR Workshop (2017)
2. Fang, Z., Sims, C.R.: Humans learn generalizable representations through efficient coding. *Nature Communications* **16**(1), 3989 (2025). <https://doi.org/10.1038/s41467-025-58848-6>
3. Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., Davidson, J.: Learning latent dynamics for planning from pixels. ICML (2019)
4. Hafner, D., Lillicrap, T., Norouzi, M., Ba, J.: Dream to control: Learning behaviors by latent imagination. ICLR (2020)
5. Hafner, D., Pasukonis, J., Ba, J., Lillicrap, T.: Mastering diverse domains through world models. ICML (2023)

6. Hansen, N., Su, H., Wang, X.: TD-MPC2: Scalable, robust world models for continuous control. ICLR (2024)
7. Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., Lerchner, A.: beta-VAE: Learning basic visual concepts with a constrained variational framework. ICLR (2017)
8. Janner, M., Li, Q., Levine, S.: Offline reinforcement learning as one big sequence modeling problem. NeurIPS (2021)
9. Kaiser, L., et al.: Model-based reinforcement learning for Atari. ICLR (2019)
10. Koh, P.W., et al.: Concept bottleneck models. ICML (2020)
11. Levine, S., Finn, C., Darrell, T., Abbeel, P.: End-to-end training of deep visuomotor policies. JMLR (2016)
12. Mentzer, F., Foster, D., Minnen, D., Agustsson, E., Toderici, G.: Finite scalar quantization: VQ-VAE made simple. ICLR (2024)
13. Micheli, V., Alonso, E., Fleuret, F.: Transformers are sample-efficient world models. ICLR (2023)
14. Chevalier-Boisvert, M., et al.: Minigrid & Miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. NeurIPS Datasets and Benchmarks (2023)
15. Mnih, V., et al.: Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015)
16. Razavi, A., van den Oord, A., Vinyals, O.: Generating diverse high-fidelity images with VQ-VAE-2. NeurIPS (2019)
17. Reed, S., et al.: A generalist agent. TMLR (2022)
18. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv:1707.06347 (2017)
19. Doshi-Velez, F., Kim, B.: Towards a rigorous science of interpretable machine learning. arXiv:1702.08608 (2017)
20. van den Oord, A., Vinyals, O., Kavukcuoglu, K.: Neural discrete representation learning. NeurIPS (2017)
21. Locatello, F., Weissenborn, D., Unterthiner, T., Mahendran, A., Heigold, G., Uszkoreit, J., Dosovitskiy, A., Kipf, T.: Object-centric learning with slot attention. NeurIPS (2020)

A Hyperparameter Tables

Table 5 lists the hyperparameters used for encoder pre-training. Table 6 lists the PPO hyperparameters used for policy learning across all conditions.

Table 5. Encoder pre-training hyperparameters.

Hyperparameter	Value
Batch size	128
Epochs	80
Learning rate (Adam)	3×10^{-3}
Commitment loss coefficient (β)	0.10
EMA decay	0.99
Dead-code threshold	1.0
Codebook size K (agent head)	64
Codebook size K (key head)	16
Codebook size K (door head)	8
Embedding dimension D_{model}	64
Transformer heads	4
Transformer layers	2
Position embedding dimension	32

Table 6. PPO policy learning hyperparameters (all conditions).

Hyperparameter	Value
n_steps	512
Batch size	64
n_epochs (per rollout)	4
Discount factor γ	0.99
GAE lambda	0.95
Clip epsilon	0.2
Entropy coefficient	0.05
Value function coefficient	0.5
Max gradient norm	0.5
Learning rate	3×10^{-3}
Total environment steps	200,000
Evaluation frequency (steps)	5,000
Evaluation episodes per checkpoint	10
Number of seeds	5
Policy net architecture	[64, 64] MLP, ReLU